Project Report

Smart Document Scanner & Analyzer

Winter Semester 2025-2026

Slot: D11+D12

CSE3010 — Computer Vision Bring
Your Own Project (BYOP)

Course Project Report March
2026

Submitted By :
Riya Kinhikar (23BAI11085)

Submitted On: 31st March, 2026 Submitted To: Dr.Amrita Parashar

Table of Contents

1.	Introduction and Problem Statement
2.	Motivation and Real-World Relevance
3.	Technical Approach
4.	System Architecture
5.	Implementation Details
6.	Course Concepts Applied
7.	Challenges Faced and Solutions
8.	Results and Observations
9.	Limitations and Future Work
10.	Conclusion and Reflections

1. Introduction and Problem Statement:

Every day, millions of people take photographs of documents using their smartphones. These include receipts, handwritten notes, printed letters, certificates, whiteboard content, and more. These photographs often have perspective distortion because the camera is rarely perfectly perpendicular to the document. They also suffer from uneven lighting, background clutter, and noise. As a result, the images can be hard to read, share, or organize effectively. Commercial document scanning apps like CamScanner and Adobe Scan effectively address these issues, but their inner workings are unclear to most users.

This project aims to create a complete document scanning and analysis tool from scratch. It will use only classical Computer Vision techniques studied in CSE3010. The purpose is to understand and showcase the algorithms that make this everyday convenience possible. The project will be built as a command-line tool. It will accept a photograph of a document and produce a clean, perspective-corrected scan. In addition to scanning, it will also conduct a detailed visual analysis.
It will include feature extraction using various algorithms like SIFT, ORB, HOG, and Harris, as well as image segmentation methods such as K-Means clustering, Watershed, and text region detection. This will show a wide range of course concepts within a single, cohesive application.

2. Motivation and Real-World Relevance

The motivation for choosing this project comes from a simple observation: document scanning is one of the most relatable uses of Computer Vision. Unlike specialized tasks such as autonomous driving or medical imaging, almost everyone has needed to scan documents at some point. By building this tool, the abstract ideas from the course syllabus, including projective transformations, edge detection, feature descriptors, and segmentation, become tied to a practical application.

Additionally, the problem is complex enough to require techniques from nearly every module in the course. Image preprocessing (Module 1) addresses noise and contrast. Edge detection and feature extraction (Module 3) pinpoint the document boundary and its structural elements. Homography and perspective correction (Modules 1 and 2) fix the geometry. Segmentation and clustering (Modules 3 and 4) examine the document layout. This wide coverage was a deliberate design choice.

It’s important to note that this project uses only traditional CV techniques, avoiding deep learning and pre-trained neural networks. This was a conscious decision. The goal was to thoroughly understand the basic algorithms rather than depend on black-box models. Every step of the pipeline has a clear mathematical foundation that connects directly to the course material.

3. Technical Approach

3.1 Document Scanning Pipeline

The scanning pipeline has a five-stage structure. First, the input image is converted to grayscale and smoothed with a Gaussian blur to reduce noise while keeping edge details. The size of the Gaussian kernel can be adjusted; larger kernels give stronger smoothing but may blur important edges. For typical smartphone photos, a 5x5 kernel offers a good balance.

Second, Canny edge detection finds pixels with strong gradient changes. The Canny algorithm uses Sobel filters to calculate gradient magnitude and direction at each pixel. It then applies non-maximum suppression to thin edges to a single pixel width, followed by hysteresis thresholding with dual thresholds. Pixels above the high threshold are marked as definite edges; those between the low and high thresholds are kept only if they connect to a strong edge. This two-threshold method creates clean, connected edge chains while filtering out isolated noise.

Third, the edge map is examined using contour detection and polygon approximation. The Douglas-Peucker algorithm simplifies each contour into a polygon with fewer vertices. The system looks for the largest quadrilateral, a four-sided polygon, that covers at least 10% of the image area. This method works because a document in a photo will usually be the largest rectangular area.

Fourth, after identifying four corner points, a 3x3 homography matrix is calculated using OpenCV's getPerspectiveTransform function. This matrix represents the projective transformation that maps the detected trapezoid to a rectangle. The warpPerspective function then uses this transformation to produce a top-down "bird's eye" view of the document, effectively removing perspective distortion.

Finally, the corrected image is improved and binarized. CLAHE (Contrast Limited Adaptive Histogram Equalization) enhances local contrast by dividing the image into tiles and equalizing each one separately. Adaptive Gaussian thresholding then converts the image to black-and-white using locally computed thresholds, making the result resilient to shadows and uneven lighting.

3.2 Feature Extraction

The feature extraction module shows four different algorithms. ORB (Oriented FAST and Rotated BRIEF) combines the FAST keypoint detector with the BRIEF binary descriptor. FAST finds corners by looking at a ring of 16 pixels around each candidate. BRIEF creates a compact binary descriptor through intensity comparisons. ORB also adds rotation invariance by calculating keypoint orientation from intensity centroids.

SIFT (Scale-Invariant Feature Transform) works through a multi-stage process. It begins with scale-space extrema detection using Difference of Gaussians (DoG). Then, it localizes keypoints with sub-pixel accuracy and assigns orientation based on local gradient histograms. Finally, it computes a descriptor with 128-dimensional gradient orientation histograms. Each descriptor captures the spatial arrangement of gradients in the keypoint area, making it robust to changes in scale, rotation, and partial illumination.

HOG (Histogram of Oriented Gradients) captures shape information by calculating gradient orientation histograms within a grid of cells and then normalizing over overlapping blocks. The Harris corner detector computes a response function based on eigenvalues of the structure tensor, which is the local gradient auto-correlation matrix. It classifies points as corners where both eigenvalues are large.

3.3 Image Segmentation

Three segmentation approaches are used. K-Means color clustering divides pixel values into k groups. It does this by assigning pixels to the nearest centroid and recalculating centroids as the average of each cluster. For documents, this effectively separates text (dark pixels), background (light pixels), and colored elements into different areas.

Watershed segmentation views the grayscale image as a topographic surface. It finds region boundaries by simulating flooding from marker points. It determines the sure foreground through distance transform thresholding and the sure background using morphological dilation. The watershed algorithm then creates boundaries where different flood basins meet. Text region detection includes adaptive thresholding and morphological closing to combine nearby characters into continuous blocks. It uses connected component analysis to identify and outline text regions with rectangles.

4. System Architecture

The project follows a modular architecture with a clear separation of concerns. The codebase is organized into four source modules in the src/ directory, each handling a specific aspect of the pipeline.

The scanner module (scanner.py) implements the core five-stage scanning pipeline: preprocessing, edge detection, contour detection, perspective transformation, and adaptive thresholding. The features module (features.py) provides ORB, SIFT, HOG, and Harris corner detection with a unified interface. The segmentation module (segmentation.py) includes K-Means clustering, Watershed segmentation, region growing, and text region detection. The visualization module (visualization.py) creates annotated output images for each pipeline stage.

The main.py entry point uses Python's argparse library to provide three CLI commands. The 'demo' command generates a synthetic test image and runs the full pipeline. The 'scan' command runs the scanning pipeline on a user-provided image. The 'analyze' command performs feature extraction and segmentation. This design allows each component to be used independently or as part of the complete pipeline.

5. Implementation Details

5.1 Point Ordering for Homography

A key detail in implementation is the consistent ordering of the four corner points before calculating the homography. The contour detection step returns corners in a random order, but the perspective transform needs a clear match between source and destination points. The solution uses the sum and difference of coordinates. The top-left corner has the smallest x+y sum, the bottom-right corner has the largest, the top-right corner has the smallest y-x difference, and the bottom-left corner has the largest. This geometric property is true no matter how the document is oriented, and it provides a reliable order without complicated angle calculations.

5.2 Graceful Fallback Handling

Not every input image has a clearly defined document. When the contour detector cannot find a quadrilateral, such as when the document takes up the whole frame or has rounded corners, the system processes the full image instead of failing. It does this by using the image corners as the "detected" contour, which means it applies an identity transform. The metadata dictionary keeps track of this fallback so the user knows what happened.

5.3 Synthetic Test Image Generation

To allow testing without needing external images, the project has a function that creates a synthetic document photo. It generates a white "document" with text and bar charts. Then, it warps this onto a textured background using a known homography. This synthetic image features controlled perspective distortion, which is perfect for checking whether the pipeline accurately detects and corrects the document. Adding Gaussian noise to the background texture makes the scene realistic enough to test how well edge detection works.

6. Course Concepts Applied

odule	Concept	Implementation
1	Gaussian Filtering & Convolution	Preprocessing with GaussianBlur
1	Histogram Processing (CLAHE)	Adaptive contrast enhancement
1	Image Enhancement	Adaptive thresholding for scan output
1	Projective Transformation	Perspective warp via getPerspectiveTransform
1	Morphological Operations	Dilation, closing for text merging
2	Homography	3x3 perspective transform matrix
3	Canny Edge Detection	Edge map with dual thresholds
3	Harris Corner Detection	Structure tensor eigenvalue analysis
3	SIFT / DOG / Scale-Space	Scale-invariant feature descriptors
3	HOG	Gradient orientation histograms
3	Region Growing	Seed-based intensity segmentation
4	K-Means Clustering	Color-based pixel clustering
4	Watershed Segmentation	Topographic flooding for boundaries
4	Object Detection	Text region bounding boxes

7. Challenges Faced and Solutions

7.1 Edge Detection Sensitivity

The initial implementation used fixed Canny thresholds of 100 and 200. These worked well for high-contrast documents but struggled with images that had soft edges or low contrast between the document and background. The solution was to use more lenient thresholds of 50 and 150, combined with morphological dilation to close small gaps in the edge contour. This trade-off introduced more noisy edges, but it made the quadrilateral detection more reliable since the contour was more likely to be complete.

7.2 Contour Approximation Epsilon

The Douglas-Peucker approximation parameter, epsilon, determines how much contours are simplified. A value that is too small creates polygons with too many vertices. A value that is too large oversimplifies and can turn a near-rectangle into a triangle. Through testing, 2% of the contour perimeter was selected as the epsilon value. This choice gives reliable four-point approximations for rectangular documents while allowing for slight curves or damage at the edges.

7.3 Handling Non-Rectangular Documents

Some documents do not appear as clean quadrilaterals. For example, a receipt may be curved, or a note may have torn edges. The system deals with this using the minimum area threshold, which is 10% of the image area, and a fallback mechanism. If no quadrilateral meets the criteria, the system processes the entire image. Although this does not give a perspective-corrected result, it ensures that the feature extraction and segmentation stages still offer useful analysis.

8. Results and Observations

The scanning pipeline was tested on both synthetic and real-world document photographs. In the synthetic test image, the system accurately identifies all four corners and creates a clean, perspective-corrected scan. The contour is detected reliably, and the homography maps the trapezoid to a rectangle correctly. The adaptive thresholding produces clear binary output with well-rendered text.

Feature extraction results show that SIFT and ORB both detect hundreds of keypoints focused around text and graphical elements, which are the high-frequency areas of the document. Harris corner detection identifies thousands of corner responses, mainly at character corners and intersections. The HOG descriptor yields a 1764-dimensional feature vector that represents the overall shape structure.

Segmentation analysis demonstrates that K-Means clustering with k=3 effectively separates text, background, and graphical elements into different regions. Watershed segmentation successfully finds boundaries between adjacent text blocks and figures. Text region detection properly identifies and bounds major text blocks with rectangular boxes, although very small text or isolated characters may be missed during the morphological merging step.

The complete demo pipeline generates 14 output images, showing the contribution of each stage visually. Processing a typical image takes less than 2 seconds on a modern machine, with feature extraction being the most computationally demanding stage due to the SIFT scale-space computation.

9. Limitations and Future Work

The current implementation has several known limitations. First, it assumes there is only one document per image. Detecting multiple documents would require a more advanced contour selection strategy that evaluates all candidate quadrilaterals and returns more than just the largest one. Second, the system depends on the document having a clear boundary; documents on white surfaces or with similar background colors may go undetected. Third, the scanning process is purely geometric and does not perform OCR (Optical Character Recognition), so it does not extract the text as machine-readable data.

Possible extensions include integrating Tesseract OCR for text extraction from scanned documents, adding optical flow analysis (Module 4) for tracking documents in videos, implementing feature matching between document templates and scanned images for automatic document classification, and incorporating Shape from Texture concepts from Module 5 to estimate the curvature of document surfaces for non-flat documents.

10. Conclusion and Reflections

The main objectives of this project were to demonstrate that the fundamental Computer Vision techniques learned in CSE3010 can work together to create a usable and functional document scanning application. The entire pipeline used the following techniques: Gaussian filtering and Canny Edge Detection from Module 1/3, Homography-Based Perspective Correction from Module 2, SIFT/ORB/HOG/Harris Feature Extraction from Module 3, and K-Means/Watershed Segmentation from Module 4. Each algorithm provides specific functions, which together create a coherent system.

The most insightful lesson was developing an understanding of how these algorithms interact with each other when they are being processed. For instance, the quality of Edge Detection affects the quality of Contour Detection, which affects the quality of corner points used to compute the Homography. Fine tuning one stage often requires going back to fine tune the previous stages — for example, the parameters that provide the best results for Edge Detection may not provide the best results for Feature Extraction. From my perspective, having this systems-level understanding of how individual algorithms integrate with one another to create a working application is the single greatest learning outcome from this project.

Building the application without using deep learning or using pre-built scanning libraries required a deep engagement with the mathematical principles underlying each technique.

Understanding why the Canny detector uses hysteresis thresholding, how the homography matrix encodes a projective transformation, and what the eigenvalues of the structure tensor signify for corner detection are insights that would be difficult to gain from using higher-level abstractions alone.